



Presentación

Blender

Miembros

Samuel J. Cruz Rosado

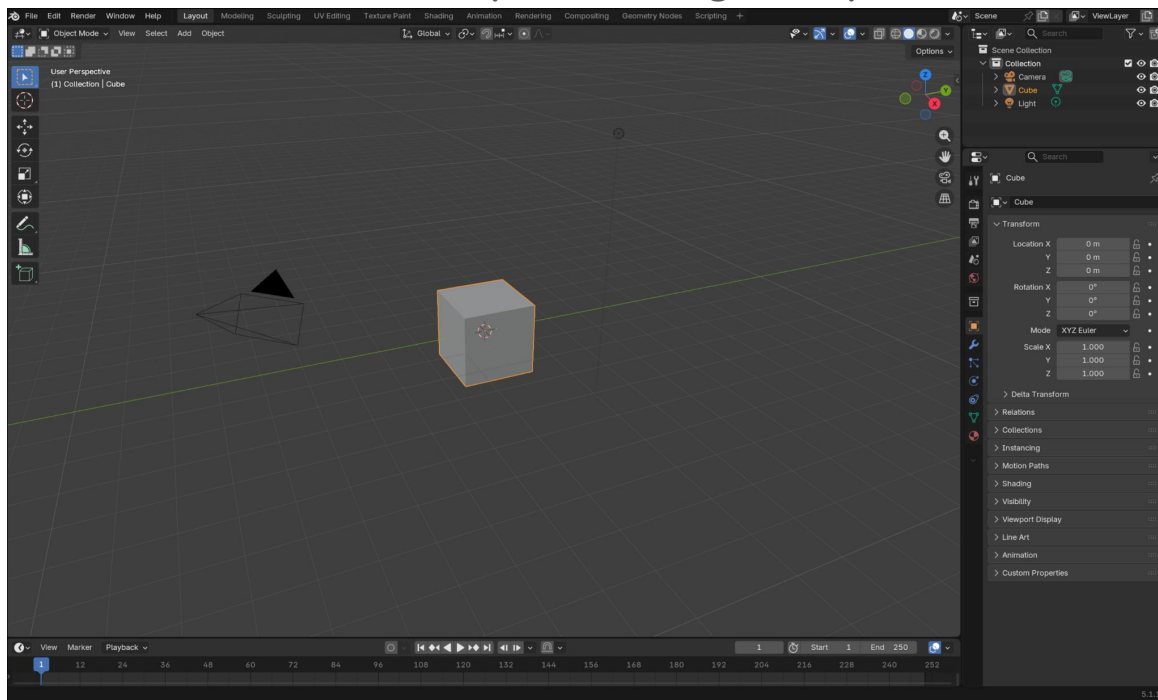
Alberto G. Rosario Tirado

Noel Y. Cordero Rivera

INTRODUCCIÓN



Blender es un programa de software libre y código abierto de creación 3D. Es una de las herramientas más potentes y completas en su categoría.





- En 1988, Ton Roosendaal fundó el estudio de animación holandés NeoGeo.
- NeoGeo se convirtió rápidamente en el estudio de animación 3D más grande de los Países Bajos y una de las casas de animación líderes en Europa.
- NeoGeo creó producciones galardonadas (European Corporate Video Award 1993 y 1995) para grandes clientes corporativos como la multinacional de electrónica Philips.
- Dentro de NeoGeo, Ton fue responsable tanto de la dirección de arte como del desarrollo de software interno.



- En 1998, Ton decidió fundar una nueva compañía llamada Not a Number (NaN) como un spin-off de NeoGeo para comercializar y desarrollar Blender.
- El núcleo de NaN era el deseo de crear y distribuir una aplicación 3D compacta y multiplataforma de forma gratuita.
- En ese momento, este era un concepto revolucionario ya que la mayoría de las aplicaciones 3D comerciales costaban miles de dólares.
- NaN esperaba poner herramientas de animación y modelado 3D de nivel profesional al alcance del público informático en general.
- El modelo comercial de NaN consistía en proporcionar productos y servicios comerciales en torno a Blender.



- Tras el éxito de la conferencia SIGGRAPH a principios de 2000, NaN obtuvo una financiación de 4,5 Millones de euros de capitalistas de riesgo.
- Esta gran entrada de efectivo permitió a NaN expandir rápidamente sus operaciones.
- Pronto, NaN contó con hasta 50 empleados que trabajaban en todo el mundo tratando de mejorar y promover Blender.
- En el verano de 2000, se lanzó Blender 2.0. Esta versión de Blender agregó la integración de un motor de juego a la aplicación 3D.
- A finales de 2000, el número de usuarios registrados en el sitio web de NaN superaba los 250,000.

CARACTERÍSTICAS



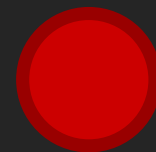
1. **Modelado y Esculturas 3D:** Herramientas robustas como soporte N-Gon, edición proporcional, escultura con múltiples pinceles y topología dinámica.
2. **Motores de Renderizados:** Incluye Cycles (trazado de rayos fotorrealista) y EeVee (motor en tiempo real ultrarrápido).
3. **Animación y Rigging:** Herramientas avanzadas para personajes, incluyendo huesos B-spline, animación no lineal (NLA) y cinemática inversa/ directa (IK/FK).
4. **Grease Pencil:** Herramienta única para animación 2D dentro del entorno 3D.
5. **Edición de Video:** Incluye un editor de video básico para cortes y composición.

VENTAJAS DE BLENDER



1. Es 100% libre, licenciado bajo [GNU GPL](#) (licencia permisiva de software libre), lo que lo hace ideal para artistas independientes y pequeños estudios.
2. Permite realizar todo el proceso de producción 3D modelado, esculturas, texturizado, rigging, animación, renderizado, composición e incluso edición de video en un mismo programa.
3. Al ser tan popular, existen miles de tutoriales gratuitos, foros de soporte y addons (complementos) desarrollados por la comunidad.
4. Funciona en Windows, macOS y Linux, y es cada vez más reconocido en la industria profesional.

DESVENTAJAS DE BLENDER



1. **Uso avanzado:** Su interfaz y flujo de trabajo pueden resultar abrumadores y difíciles para principiantes en comparación con otros programas.
2. **Estandarización en la Industria:** Aunque su uso crece, todavía no es el estándar principal en grandes estudios de cine o videojuegos, donde suelen preferirse herramientas como Autodesk Maya o Autodesk 3ds Max.
3. **Rendimiento en Escenas Complejas:** Pueden tener limitaciones de rendimientos frente a software muy especializado en simulaciones físicas, de fluidos o animaciones faciales de altísima complejidad.
4. **Evaluación no-estandarizada:** Blender está en constante desarrollo, lo cual significa que su flujo de trabajo puede cambiar, haciendo que usuarios tengan que aprender nuevos atajos o cambiar su forma de trabajar comparado con otros programas.

PROGRAMACIÓN



Blender tiene un módulo llamado “bpy” (Blender Python), la cual permite el control total de Blender y sus funciones de forma programática utilizando una versión integrada de Python. Su incorporación fue necesaria para reducir las tareas repetitivas y permitir la integración de extensiones (add-ons) a Blender, extendiendo el funcionamiento. Básicamente si una función no existe en Blender, la comunidad o incluso tu la puedes crear.

<https://pypi.org/project/bpy/>

PROGRAMACIÓN



Función “helix” con
expansión logarítmica.

Fórmula original

$$s = 5$$

$$v = 3$$

$$x(i) = s \cdot \sin(i) \cdot (1 - \log_3(i))$$

$$z(i) = s \cdot \cos(i) \cdot (1 - \log_3(i))$$

$$t(i) = v \cdot i$$

```
# Creado por Samuel J. Cruz Rosado
```

```
import bpy  
import math
```

```
bpy.ops.object.select_all(action='SELECT')  
bpy.ops.object.delete()
```

```
objs = []
```

```
scale = 5  
vertical_scale = 3
```

```
for i in range(1,50):  
    decay = 1 - math.log(i, 3)
```

```
    x = math.sin(i) * scale * decay  
    z = math.cos(i) * scale * decay  
    t = i * vertical_scale
```

```
    bpy.ops.mesh.primitive_uv_sphere_add(location=(x, z, i))  
    object = bpy.context.active_object  
    objs.append(object)
```

```
for obj in objs:  
    obj.select_set(True)
```

```
bpy.ops.object.join()
```

Samuel J. Cruz Rosado

PROGRAMACIÓN



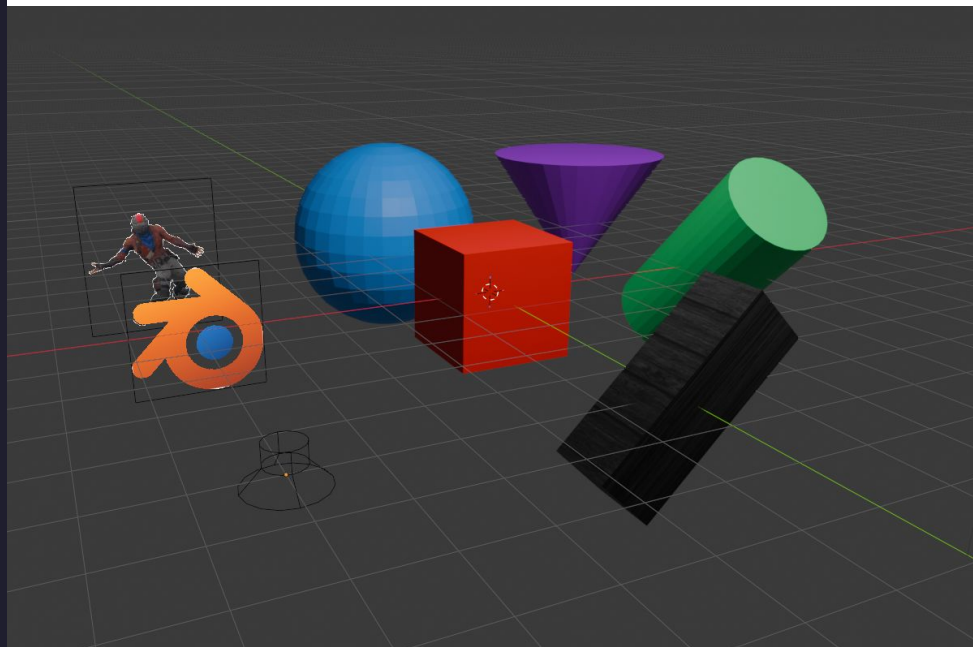
Resultado:





Primitivos

```
2 def main() → None:
3     clear_objects()
4
5     mesh.primitive_cube_add(size=2, location=(0, 0, 0))
6     apply_color("Red", (1.0, 0.0, 0.0))
7
8     mesh.primitive_cube_add(
9         size=1, location=(0, -5, 0), scale=(3, 1, 1), rotation=(0, -pi / 4, 0)
10    )
11    apply_texture("Wood", "./wood.jpg")
12
13    mesh.primitive_uv_sphere_add(radius=2, location=(0, 5, 0))
14    apply_color("Blue", (0.0, 0.3, 1.0))
15
16    mesh.primitive_cylinder_add(
17        radius=1, depth=4, location=(5, 0, 0), rotation=(pi / 4, 0, 0)
18    )
19    apply_color("Green", (0.0, 0.8, 0.2))
20
21    mesh.primitive_cone_add(
22        radius1=2, radius2=0, depth=3, location=(5, 5, 0), rotation=(pi, 0, 0)
23    )
24    apply_color("Purple", (0.5, 0.0, 1.0))
25
26    add_image("./logo.png", location=(-5, 0, 0), size=2)
27    add_video("./guest.gif", location=(-5, 5, 0), size=3)
28    add_audio("./sfx.mp3", location=(-5, -5, 0))
29
30    object.select_all(action="DESELECT")
31
32
33 if __name__ == "__main__":
34     main()
```





Box (Cubo)

```
import bpy
from math import pi

# Cubo
bpy.ops.mesh.primitive_cube_add(
    size=2,
    location=(0,0,0) # Traslacion
)
# Rectángulo
bpy.ops.mesh.primitive_cube_add(
    size=1,
    location=(0,0,0), # Traslacion
    scale=(3,1,1) # Escala
    rotation=(0,-pi/4,0) # Rotación (Euler)
)
```

```
bpy.ops.mesh.primitive_cube_add(*, size=2.0, calc_uvns=True, enter_editmode=False,
    align='WORLD', location=(0.0, 0.0, 0.0), rotation=(0.0, 0.0, 0.0), scale=(0.0,
    0.0, 0.0))
```

Construct a cube mesh that consists of six square faces

- PARAMETERS:**
- **size** (*float*) – Size, (in [0, inf], optional)
 - **calc_uvns** (*bool*) – Generate UVs, Generate a default UV map (optional)
 - **enter_editmode** (*bool*) – Enter Edit Mode, Enter edit mode when adding this object (optional)
 - **align** (*Literal*['WORLD', 'VIEW', 'CURSOR']) – Align, The alignment of the new object (optional)
 - **WORLD** World – Align the new object to the world.
 - **VIEW** View – Align the new object to the view.
 - **CURSOR** 3D Cursor – Use the 3D cursor orientation for the new object.
 - **location** (*mathutils.Vector*) – Location, Location for the newly added object (array of 3 items, in [-inf, inf], optional)
 - **rotation** (*mathutils.Euler*) – Rotation, Rotation for the newly added object (array of 3 items, in [-inf, inf], optional)
 - **scale** (*mathutils.Vector*) – Scale, Scale for the newly added object (array of 3 items, in [-inf, inf], optional)

RETURNS: Result of the operator call.

RETURN TYPE: set[Literal[Operator Return Items]]



Cone

```
import bpy
from math import pi

bpy.ops.mesh.primitive_cone_add(
    radius1=2, # Radio base
    radius2=0, # Radio punta
    depth=3, # Profundidad
    location=(5,5,0), # Traslacion
    rotation=(pi,0,0) # Rotación (Euler)
)
```

```
bpy.ops.mesh.primitive_cone_add(*, vertices=32, radius1=1.0, radius2=0.0, depth=2.0,
    end_fill_type='NGON', calc_uv=True, enter_editmode=False, align='WORLD',
    location=(0.0, 0.0, 0.0), rotation=(0.0, 0.0, 0.0), scale=(0.0, 0.0, 0.0))
```

Construct a conic mesh

PARAMETERS:

- **vertices** (*int*) – Vertices, (in [3, 10000000]), optional
- **radius1** (*float*) – Radius 1, (in [0, inf], optional)
- **radius2** (*float*) – Radius 2, (in [0, inf], optional)
- **depth** (*float*) – Depth, (in [0, inf], optional)
- **end_fill_type** (*Literal*['NOTHING', 'NGON', 'TRIFAN']) – Base Fill Type, (optional)
 - **NOTHING** Nothing – Don't fill at all.
 - **NGON** N-Gon – Use n-gons.
 - **TRIFAN** Triangle Fan – Use triangle fans.
- **calc_uv** (*bool*) – Generate UVs, Generate a default UV map (optional)
- **enter_editmode** (*bool*) – Enter Edit Mode, Enter edit mode when adding this object (optional)
- **align** (*Literal*['WORLD', 'VIEW', 'CURSOR']) – Align, The alignment of the new object (optional)
 - **WORLD** World – Align the new object to the world.
 - **VIEW** View – Align the new object to the view.
 - **CURSOR** 3D Cursor – Use the 3D cursor orientation for the new object.
- **location** (*mathutils.Vector*) – Location, Location for the newly added object (array of 3 items, in [-inf, inf], optional)
- **rotation** (*mathutils.Euler*) – Rotation, Rotation for the newly added object (array of 3 items, in [-inf, inf], optional)
- **scale** (*mathutils.Vector*) – Scale, Scale for the newly added object (array of 3 items, in [-inf, inf], optional)

RETURNS: Result of the operator call.

RETURN TYPE: set[Literal[Operator Return Items]]



Sphere (UV Sphere)

```
import bpy
```

```
bpy.ops.mesh.primitive_uv_sphere_add(  
    radius=2, # Radio  
    location=(0,5,0) # Traslacion  
)
```

```
bpy.ops.mesh.primitive_uv_sphere_add(*, segments=32, ring_count=16, radius=1.0,  
    calc_uv=True, enter_editmode=False, align='WORLD', location=(0.0, 0.0, 0.0),  
    rotation=(0.0, 0.0, 0.0), scale=(0.0, 0.0, 0.0))
```

Construct a spherical mesh with quad faces, except for triangle faces at the top and bottom

PARAMETERS:

- **segments** (*int*) – Segments, (in [3, 100000], optional)
- **ring_count** (*int*) – Rings, (in [3, 100000], optional)
- **radius** (*float*) – Radius, (in [0, inf], optional)
- **calc_uv** (*bool*) – Generate UVs, Generate a default UV map (optional)
- **enter_editmode** (*bool*) – Enter Edit Mode, Enter edit mode when adding this object (optional)
- **align** (*Literal*['WORLD', 'VIEW', 'CURSOR']) – Align, The alignment of the new object (optional)
 - **WORLD** World – Align the new object to the world.
 - **VIEW** View – Align the new object to the view.
 - **CURSOR** 3D Cursor – Use the 3D cursor orientation for the new object.
- **location** (*mathutils.Vector*) – Location, Location for the newly added object (array of 3 items, in [-inf, inf], optional)
- **rotation** (*mathutils.Euler*) – Rotation, Rotation for the newly added object (array of 3 items, in [-inf, inf], optional)
- **scale** (*mathutils.Vector*) – Scale, Scale for the newly added object (array of 3 items, in [-inf, inf], optional)

RETURNS: Result of the operator call.

RETURN TYPE: set[Literal[Operator Return Items]]



Cylinder

```
import bpy
from math import pi

bpy.ops.mesh.primitive_cylinder_add(
    radius=1, # Radio
    depth=4, # Altura
    location=(5,0,0), # Traslacion
    rotation=(pi/4,0,0) # Rotación (Euler)
)
```

```
bpy.ops.mesh.primitive_cylinder_add(*, vertices=32, radius=1.0, depth=2.0,
end_fill_type='NGON', calc_uv=True, enter_editmode=False, align='WORLD',
location=(0.0, 0.0, 0.0), rotation=(0.0, 0.0, 0.0), scale=(0.0, 0.0, 0.0))
```

Construct a cylinder mesh

- PARAMETERS:**
- **vertices** (*int*) – Vertices, (in [3, 1000000]), optional
 - **radius** (*float*) – Radius, (in [0, inf]), optional
 - **depth** (*float*) – Depth, (in [0, inf]), optional
 - **end_fill_type** (*Literal*['NOTHING', 'NGON', 'TRIFAN']) – Cap Fill Type, (optional)
 - **NOTHING** Nothing – Don't fill at all.
 - **NGON** N-Gon – Use n-gons.
 - **TRIFAN** Triangle Fan – Use triangle fans.
 - **calc_uv** (*bool*) – Generate UVs, Generate a default UV map (optional)
 - **enter_editmode** (*bool*) – Enter Edit Mode, Enter edit mode when adding this object (optional)
 - **align** (*Literal*['WORLD', 'VIEW', 'CURSOR']) – Align, The alignment of the new object (optional)
 - **WORLD** World – Align the new object to the world.
 - **VIEW** View – Align the new object to the view.
 - **CURSOR** 3D Cursor – Use the 3D cursor orientation for the new object.
 - **location** (*mathutils.Vector*) – Location, Location for the newly added object (array of 3 items, in [-inf, inf], optional)
 - **rotation** (*mathutils.Euler*) – Rotation, Rotation for the newly added object (array of 3 items, in [-inf, inf], optional)
 - **scale** (*mathutils.Vector*) – Scale, Scale for the newly added object (array of 3 items, in [-inf, inf], optional)

RETURNS: Result of the operator call.

RETURN TYPE: set[Literal[Operator Return Items]]



Texturas

```
import bpy
from math import pi

def apply_texture(name: str, filepath: str) → None:

    mat = bpy.data.materials.new(name=name)
    mat.use_nodes = True
    nodes = mat.node_tree.nodes
    links = mat.node_tree.links

    bsdf = nodes["Principled BSDF"]
    tex_node = nodes.new("ShaderNodeTexImage")
    tex_node.image = bpy.data.images.load(filepath)

    links.new(tex_node.outputs["Color"], bsdf.inputs["Base Color"])
    context.active_object.data.materials.append(mat)
```

```
bpy.ops.mesh.primitive_cube_add(
    size=1,
    location=(0,0,0),
    scale=(3,1,1)
    rotation=(0,-pi/4,0)
)
# Aplicas la textura debajo
# del primitivo creado
apply_texture("Wood", "./wood.jpg")
# apply_texture("Video", "./video.mp4")
```

[Link del video](#)



Colores

```
import bpy
from math import pi
def apply_color(name: str, color: tuple[float, float, float]) → None:
    mat = bpy.data.materials.new(name=name)
    bsdf = mat.node_tree.nodes["Principled BSDF"]
    bsdf.inputs["Base Color"].default_value = (*color, 1.0)
    mat.diffuse_color = (*color, 1.0)
    context.active_object.data.materials.append(mat)
```

```
bpy.ops.mesh.primitive_cube_add(
    size=1,
    location=(0,0,0),
    scale=(3,1,1)
    rotation=(0,-pi/4,0)
)
# Aplicas el color debajo del primitivo
# creado
apply_color("Red", (1,0,0))
```



Audio

```
import bpy
```

```
object.speaker_add(location=(0,0,0))
```

```
spk = context.active_object.data
```

```
spk.sound = bpy.data.sounds.load("audio.wav")
```

[Link del audio](#)

```
bpy.ops.object.speaker_add(*, enter_editmode=False, align='WORLD', location=(0.0, 0.0, 0.0), rotation=(0.0, 0.0, 0.0), scale=(0.0, 0.0, 0.0))
```

Add a speaker object to the scene

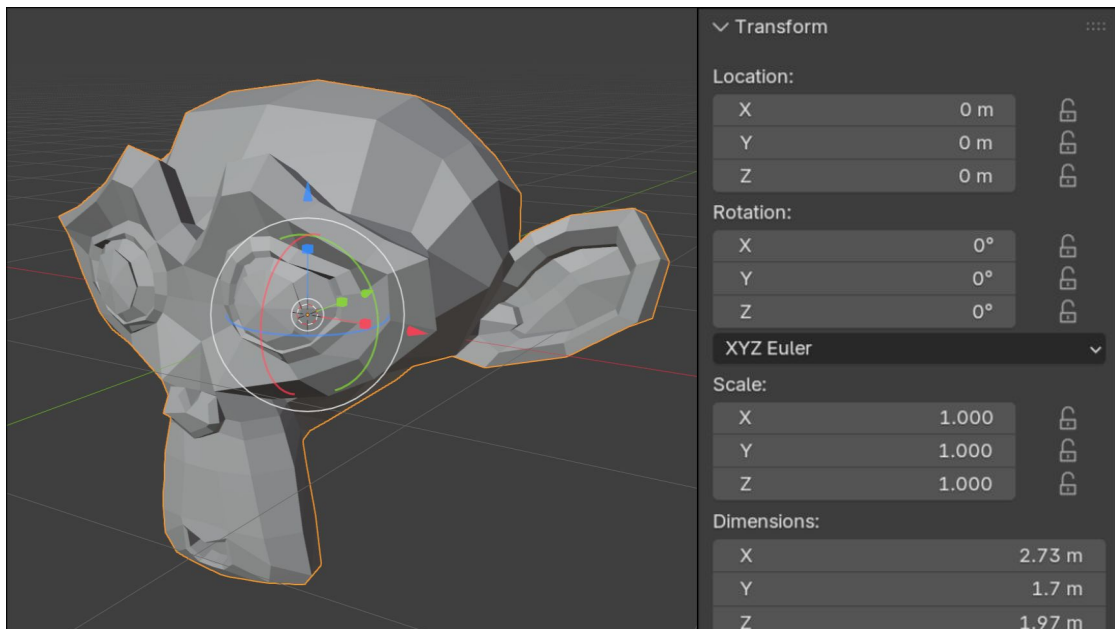
- PARAMETERS:**
- **enter_editmode** (*bool*) – Enter Edit Mode, Enter edit mode when adding this object (optional)
 - **align** (*Literal['WORLD', 'VIEW', 'CURSOR']*) – Align, The alignment of the new object (optional)
 - **WORLD** World – Align the new object to the world.
 - **VIEW** View – Align the new object to the view.
 - **CURSOR** 3D Cursor – Use the 3D cursor orientation for the new object.
 - **location** (*mathutils.Vector*) – Location, Location for the newly added object (array of 3 items, in [-inf, inf], optional)
 - **rotation** (*mathutils.Euler*) – Rotation, Rotation for the newly added object (array of 3 items, in [-inf, inf], optional)
 - **scale** (*mathutils.Vector*) – Scale, Scale for the newly added object (array of 3 items, in [-inf, inf], optional)

RETURNS: Result of the operator call.

RETURN TYPE: set[Literal[Operator Return Items]]



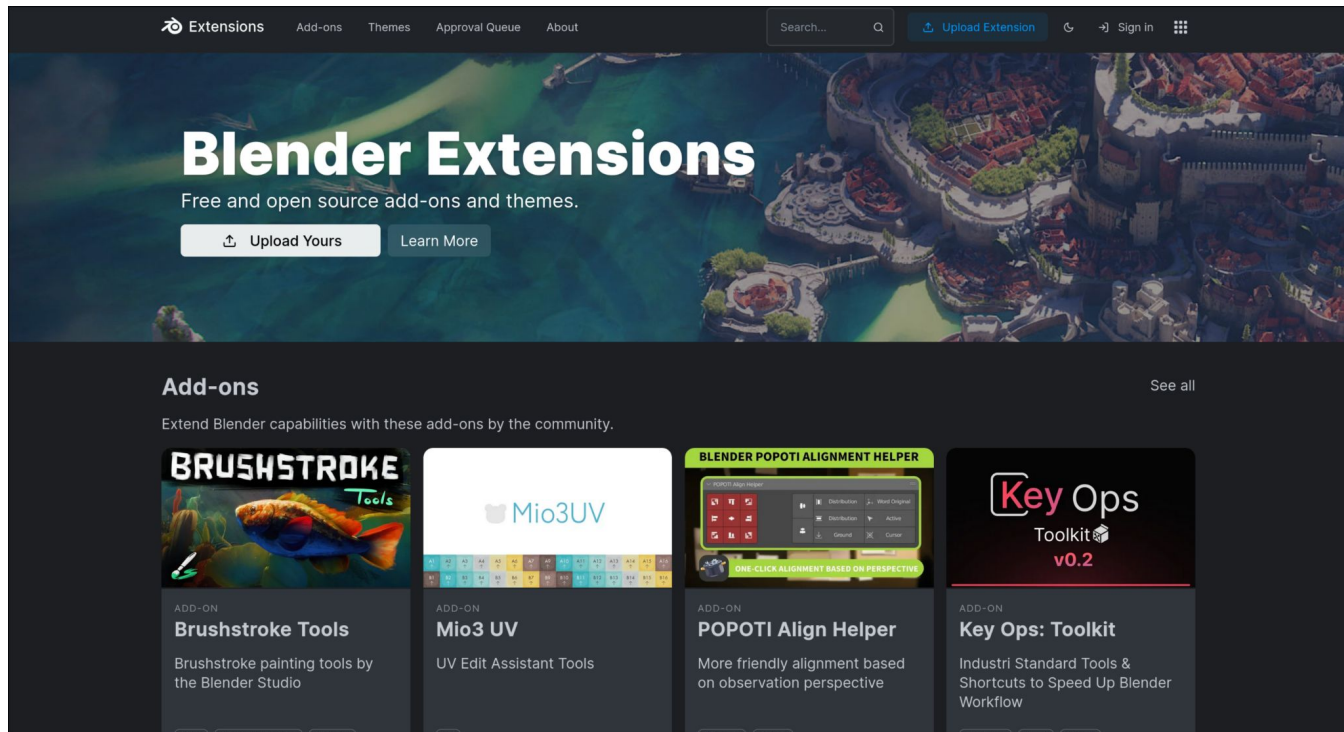
A pesar de hacer muchas cosas programáticamente, blender es un programa con interfaz de usuario gráfica, lo cual significa que traslación, rotación y escala pueden aplicarse visualmente.



PROGRAMACIÓN



A pesar de tu tener disponible la creación de programas, Blender cuenta con una biblioteca de extensiones creadas por la comunidad. Esta librería se llama Blender Extensions



<https://extensions.blender.org/>

Samuel J. Cruz Rosado

PINTAR



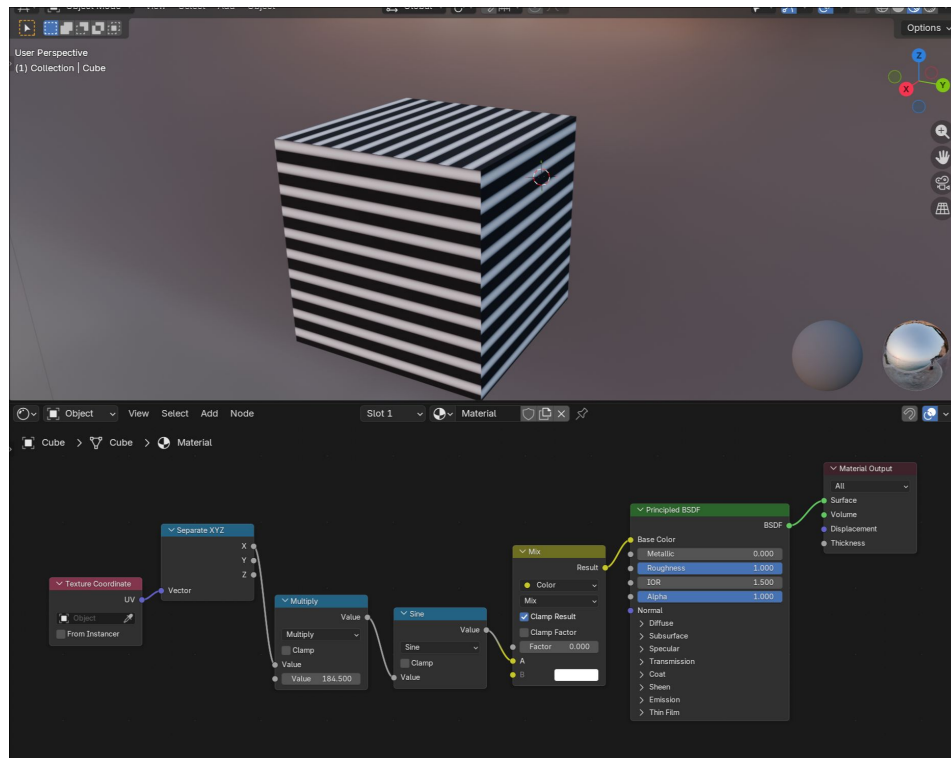
Blender cuenta con una opción de pintar la cual es muy completa, al ofrecer opciones de pinceles avanzada, la cual es pensada para aplicar color a texturas con una referencia en 3D.

Extensiones cómo ucpaint, provee una experiencia similar a Photoshop al permitir la manipulación de imágenes con capas.

NODOS (SHADERS)



Una opción muy usada de Blender es la manipulación de pixeles utilizando nodos, los cuales nos deja crear programas llamado shaders para dar color y efectos basado en las propiedades del modelo.

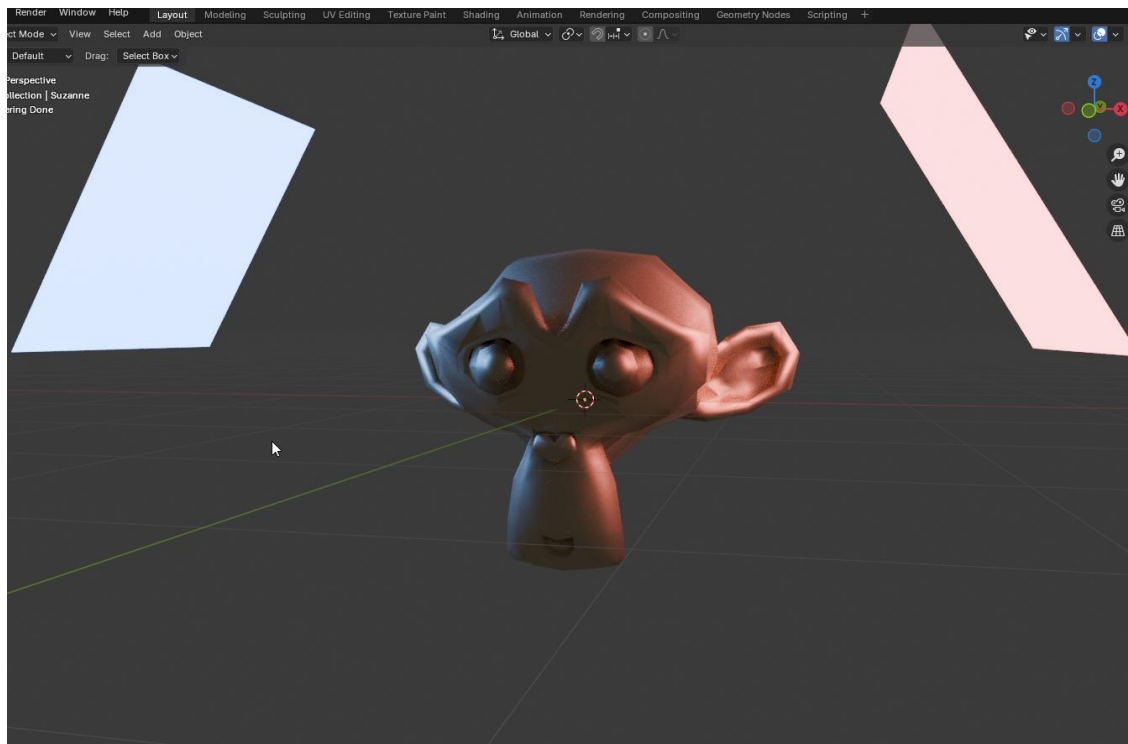


RENDERIZADO (CYCLES)

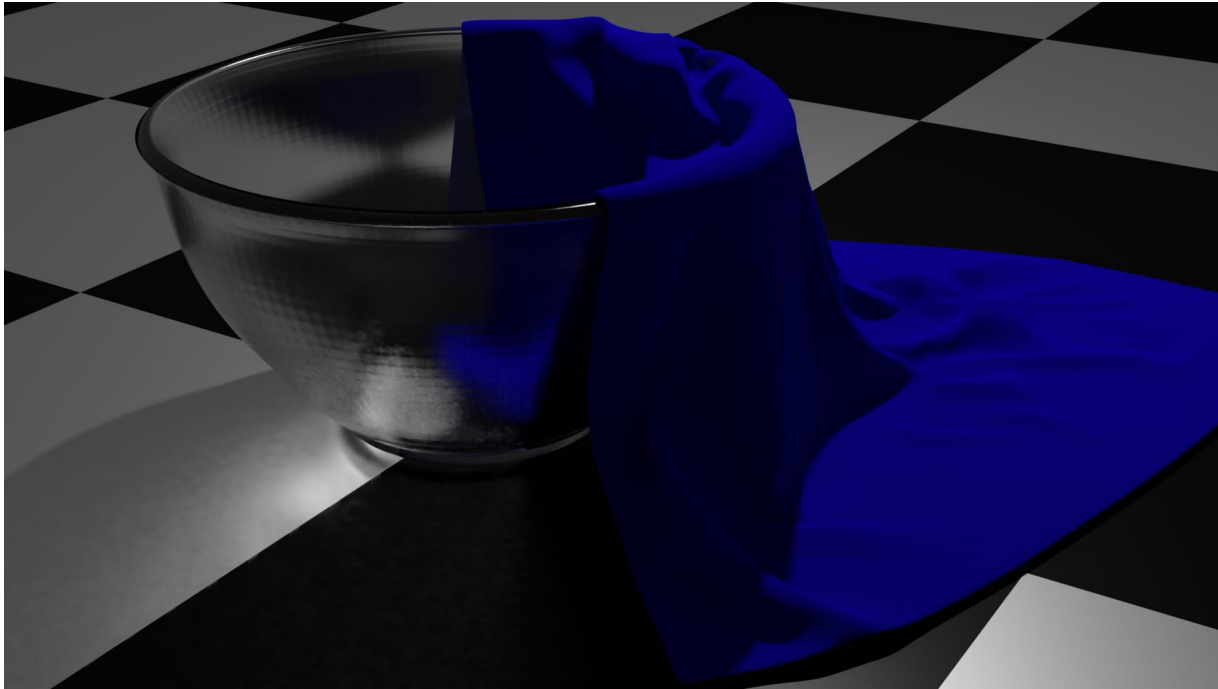


Una de las características más importantes de blender es la capacidad de renderizar escenas utilizando un motor de trazado de rayos que simula la iluminación en la vida real llamado Cycles. Esto lo pone en la misma categoría que otros programas cómo Autodesk Maya o Pixar Renderman en realismo.

RENDERIZADO (CYCLES)



RENDERIZADO (CYCLES)



Ejemplos de Producción



Blender está cada vez siendo más y más adoptado por la industria profesional, uno de los ejemplos más notables son películas cortas hechas por **Blender Foundation**, mostrando en qué estado se encuentra Blender. Blender Foundation hace este tipo de películas una vez cada 1-2 años

Ejemplos de Producción



Ejemplos de Producción



Aparte de Blender Foundation, hay películas hechas por estudios de producción reales, incluso hay películas hechas por estudios independientes o individuos, esto es lo que hace Blender tan versátil, pequeños o grandes las posibilidades son infinitas.

Ejemplos de Producción



CONCLUSIÓN



- En la industria mayormente se favorece programas ya establecidos cómo Maya, 3DS Max o Cinema 4D, estos programas suelen estar fuera del alcance de personas por su costo superando las 4 cifras
- Blender es gratis, código abierto y muy potente, lo cual permite que cualquier persona se adentre al mundo de la animación y el modelado 3D.
- Puede ser abrumador al principio, pero con practica y estudio, luego el límite se vuelve la imaginación.

MOOC's



<https://www.classcentral.com/report/best-blender-courses/>

<https://www.coursera.org/courses?query=blender>

<https://www.cgboost.com/>

REFERENCIAS



- [Historia] <https://archive.blender.org/wiki/2015/index.php/Doc:DK/2.6/Manual/>
- [Blender Python] <https://docs.blender.org/api/current/index.html>
- [Pros & Cons] <https://medium.com/@ipbsforum/blender-3d-background-advantage-and-disadvantage-bde0f86fedf7>
- [GNU GPL] <https://www.gnu.org/licenses/gpl-3.0.en.html>



Gracias por su
atencion!